



CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir



Passing Arguments to Functions

Pass by Value

Creating copies...

Pass by Value

- We have seen this before.
- Normally, single variables (and objects) will be passed by value to functions.
- This means, that their copies are created in memory, and those are used within the functions.
- Recall that arrays are **not** passed by value.
 - Instead the address of the first element is passed (**pass by address**).

```

1  #include<iostream>
2
3  void changeNumbers(int num1 = 0, int num2 = 0);
4
5  int main()
6  {
7      int x{5};
8      int y{9};
9
10     changeNumbers(x, y);
11
12     std::cout << "x = " << x << ", y = " << y << '\n';
13
14     return 0;
15 }
16
17 void changeNumbers(int num1, int num2)
18 {
19     num1 = 99;
20     num2 = 1000;
21 }

```

Note that copies of **x** and **y** are sent over to **changeNumbers()**.

I am making modifications to the copies of **x** and **y**, therefore no changes are seen on **x** and **y** after coming back to **main()**.

x

5

0x10

y

9

0x20

num1

5

0x40

num2

9

0x80

Copies

Pass by Reference

*But what is a **reference**?*

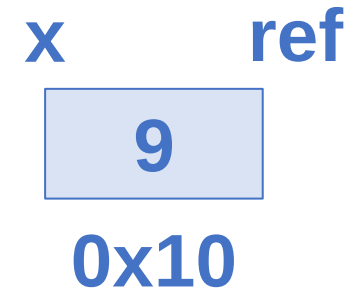
<https://www.learncpp.com/cpp-tutorial/pass-by-lvalue-reference/>

<https://www.learncpp.com/cpp-tutorial/pass-by-const-lvalue-reference/>

References in 3 Slides

- References are aliases to **existing** variables.
 - Cannot be null.
- Simply speaking, you have created another name (or identifier) for that same piece of memory.
- **ref** and **x** can be used synonymously.

```
int x{ 9 };  
int& ref{ x };
```



References in 3 Slides

```
1  #include <iostream>
2
3  int main()
4  {
5      int x { 5 };    // x is a normal integer variable
6      int& ref { x }; // ref is an lvalue reference variable that can now be used as an alias for variable x
7
8      std::cout << x << '\n'; // print the value of x (5)
9      std::cout << ref << '\n'; // print the value of x via ref (5)
10
11     return 0;
12 }
```


References in 3 Slides

- You can play around with **x** using **ref** like normal for the most part.
- Note that references are not entities themselves.

```
std::cout << &ref;      // will print &x (the address of x)
```

- Also, **ref** cannot be bound to another data once initialized.
 - Note that this is unlike pointers.

Pass by Reference

- Instead of creating copies with pass by value, you can **pass by reference**.
- No copies are created.
- The actual entity is being used directly.

```

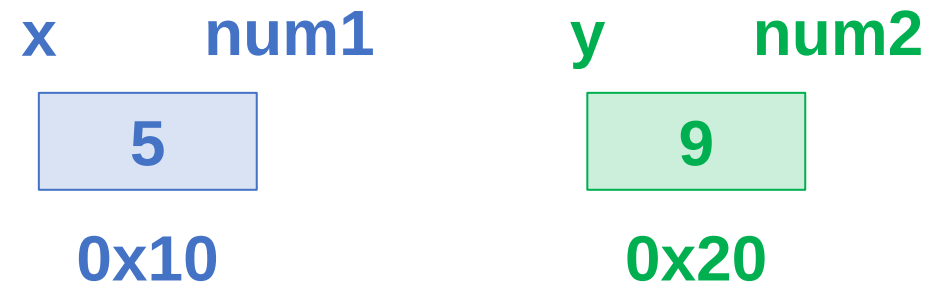
1  #include<iostream>
2
3  void changeNumbers(int& num1, int& num2);
4
5
6  int main()
7  {
8      int x{5};
9      int y{9};
10
11     changeNumbers(x, y);
12
13     std::cout << "x = " << x << ", y = " << y << '\n';
14
15
16
17     return 0;
18 }
19
20 void changeNumbers(int& num1, int& num2)
21 {
22     num1 = 99;
23     num2 = 1000;
24 }
25
26
27
28
29

```

Note that **x** and **y** are being worked upon directly by **changeNumbers()**.

I am making modifications to **x** and **y** directly. As such, we can see the changes in **main()**.

Function is called in the same way as in pass by value.



```

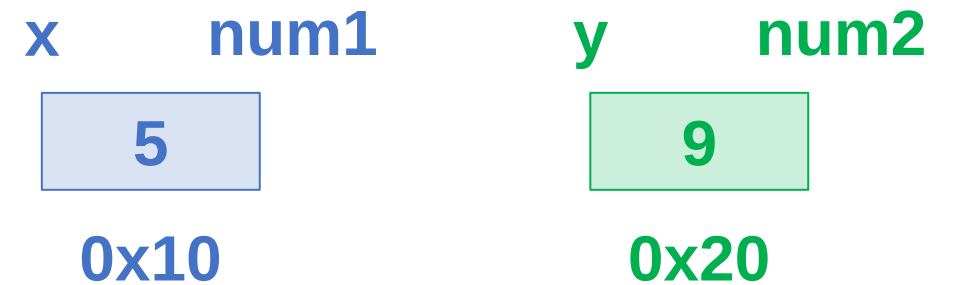
1  #include<iostream>
2
3
4  int add(const int& num1, const int& num2);
5
6  int main()
7  {
8      int x{5};
9      int y{9};
10
11
12
13
14
15      std::cout << "add(x, y) = " << add(x, y) << '\n';
16
17      return 0;
18  }
19
20
21
22
23
24
25
26  int add(const int& num1, const int& num2)
27  {
28      return num1 + num2;
29  }

```

If I don't plan on modifying the arguments, I can instead **pass by const reference**.

This will prevent any modification of **num1 (x)** and **num2 (y)** in our example.

Function is called in the same way as in pass by value.



Best practice

Favor passing by const reference over passing by non-const reference unless you have a specific reason to do otherwise (e.g. the function needs to change the value of an argument).

Pass by Reference

- Why is this helpful when we have pass by value?
- The reason is that some entities are **expensive to copy**.
 - For e.g. objects which are essentially collection of a lot of data, e.g. **std::string** type objects (which are **char** arrays), **std::vector** type objects (arrays of any data type), etc.
 - FYI, regular arrays (**dtype arr[]**) automatically decay into pointers, so they are **passed by address**.
- As we saw, references are just another name to access the same entity.
 - So no additional copy is created.

Pass by Address

Pointers again...

<https://www.learncpp.com/cpp-tutorial/pass-by-address/>

<https://www.learncpp.com/cpp-tutorial/pass-by-address-part-2/>

Pass by Address

- Instead of passing variables or their references, we pass their addresses.
 - That means we are working with pointers.
- Again, no copies of the variables are created since you are just passing the addresses.
 - Pointer variables, however, are created.
 - They are local to the function being called.


```

1  #include<iostream>
2
3  void changeNumbers(int* num1, int* num2);
4
5  int main()
6  {
7      int x{5};
8      int y{9};
9
10     changeNumbers(&x, &y);
11
12     std::cout << "x = " << x << ", y = " << y << '\n';
13
14     return 0;
15 }
16
17 void changeNumbers(int* num1, int* num2)
18 {
19     *num1 = 99;
20     *num2 = 1000;
21 }

```

Note the change in function call – addresses are being passed.

Since we have the addresses of **x** and **y**, we are essentially messing around with them directly.

Hence, the changes made in the function will be seen in **main()**.

Do you think any entity is created when **changeNumbers()** is called?

Pointers

num1

&x

0x40

num2

&y

0x80

x

5

0x10

y

9

0x20

Note

More on **const** with pointers when we talk about **this** pointer in OOP.

Pass by Address

- Just like in pass by reference, if you don't expect to modify the values of your data, you can use **pointer to const**.

- Your function declaration would look like the following.

```
void changeNumbers(const int* num1, const int* num2);
```

- This will prevent any modifications within the function, i.e. you will be unable to do the following.

```
*num1 = 1000;           // compile-time error
```

Best practice

Prefer pass by reference to pass by address unless you have a specific reason to use pass by address.

You may end up passing **nullptr** when using pass by address accidentally. So you will have to check for **nullptr** before dereferencing.

Other reasons as well: <https://www.learncpp.com/cpp-tutorial/pass-by-address/>

Discussed in class

```
#include <iostream>
```

```
void foo(int* ptr)
{
    std::cout << *ptr << '\n';
}
```

```
int main()
{
    int a{ 4 };    // created in memory (4 bytes; local to main())
    int* p{ &a };  // created in memory (8 bytes; local to main())
    foo(p);        // triggers creation of ptr in memory, initialized to the value of p (which is equal to &a) (8 bytes; local to foo())

    // 'a' is being passed to foo() by address, through pointer 'p'.
    // 'p' is being passed to foo() by value; copy is ptr, local to foo().

    return 0;
}
```

Return by Address

Return by Reference

- Return by address essentially means that your function returns an address (or a pointer).
 - You've probably been already doing this.
 - The return type of your function is a pointer type.
- Return by reference – *we will hopefully see this in OOP.*

Self Study

Came across the following link that helps give an example as to why **using namespace std;** is bad.

<https://stackoverflow.com/questions/13402789/why-does-a-swap-function-taking-two-pointers-only-work-with-using-namespace-std>